

RAPPORT DE TRAVAUX

TRAITEMENT D'IMAGES : VISION INDUSTRIELLE, TÉLÉDÉTECTION ET CARTOGRAPHIE MARINE

SeaTech, Toulon, France

Janvier 2026

Timm Claverie

Superviseure : A. Minghelli



Table des matières

1	Introduction	1
2	Vision Industrielle	2
2.1	Initialisation du modèle	2
2.2	Test du modèle	4
2.3	Analyse des résultats	5
2.4	Ouverture : Méthode du cercle	5
2.5	Bilan	6
3	Télédétection par drone	7
3.1	Initialisation	7
3.2	Différentiation et Comparaison	8
3.3	Calcul de proportion	9
3.4	Bilan	11
4	Cartographie des fonds par images sous-marines	12
4.1	Affichage des bandes et histogramme	13
4.2	Échantillonnage par région d'intérêt (ROI)	14
4.3	Classification	15
4.4	Bilan	16
5	Conclusion	17
6	Annexe	18
6.1	Code Vision Industrielle	18
6.2	Code Télédétection par Drones	21
6.3	Code Cartographie Marine	24

Résumé

Ce rapport détaille la mise en œuvre de techniques de traitement d'images appliquées à la vision industrielle, la télédétection agricole et la cartographie marine. Il présente les méthodes de segmentation par seuillage, le calcul d'indices de végétation (NDVI) et la classification supervisée via l'algorithme Spectral Angle Mapper. L'analyse des signatures spectrales et des histogrammes permet de valider la détection automatique d'objets et le suivi temporel de cultures malgré les variations d'éclairage.

Mots clés : traitement d'images, vision industrielle, télédétection agricole, cartographie marine, seuillage, NDVI, classification supervisée, signatures spectrales

This report details the implementation of image processing techniques applied to machine vision, agricultural remote sensing, and marine mapping. It presents thresholding segmentation methods, vegetation index calculation (NDVI), and supervised classification using the Spectral Angle Mapper algorithm. The analysis of spectral signatures and histograms makes it possible to validate automatic object detection and temporal monitoring of crops despite variations in lighting.

Keywords : image processing, machine vision, agricultural remote sensing, marine mapping, thresholding, NDVI, supervised classification, spectral signatures

1 Introduction

Ce rapport présente les travaux réalisés dans le cadre du cours de Traitement d'Images. L'objectif est d'explorer et de mettre en œuvre différentes techniques d'analyse d'images appliquées aux problèmes suivant : le contrôle industriel, le suivi agricole et l'étude environnementale marine.

Le document s'articule autour de trois axes principaux, illustrant une complexité croissante des méthodes de traitement :

La Vision Industrielle : La première partie se concentre sur l'automatisation du comptage d'objets, en l'occurrence de bonbons, à partir d'une banque d'images. Nous y abordons les méthodes de conversion en niveaux de gris, d'analyse d'histogrammes et de seuillage binaire pour distinguer les objets de leur arrière-plan. Cette section introduit également une méthode de correction ("méthode du cercle" $\approx$ érosion) pour affiner la précision du comptage face aux variations d'éclairage.

La Télédétection par drone : La seconde partie applique l'analyse spectrale au domaine agricole pour évaluer la croissance d'un champ de betteraves au cours du temps. En exploitant des images multispectrales (bandes visibles et infrarouges), nous calculons l'Indice de Végétation par Différence Normalisée (NDVI) pour différencier la biomasse du sol nu. Cette analyse temporelle permet de quantifier l'évolution des proportions de culture à différentes dates.

La Cartographie des fonds marins : La dernière partie traite de la classification supervisée d'images sous-marines fournies par l'IFREMER. L'objectif est de cartographier différentes classes d'objets (sable, posidonie, algue *Caulerpa Taxifolia*) en se basant sur leurs signatures radiométriques. Nous comparons ici l'efficacité de deux algorithmes de classification : la distance euclidienne et le Spectral Angle Mapper (SAM), afin de déterminer la méthode la plus robuste face aux conditions sous-marines.

L'ensemble de ces travaux, appuyés par les codes Python fournis en annexe, a été réalisé sur Anaconda sur un environnement de traitement d'image appelé "cv2".

2 Vision Industrielle

L'objectif de ce TP est de compter automatiquement des objets dans une image pour des applications de vision industrielle. Nous nous concentrerons sur une banque d'images de bonbons. Pour ce faire, nous allons utiliser le plan d'action ci-dessous :

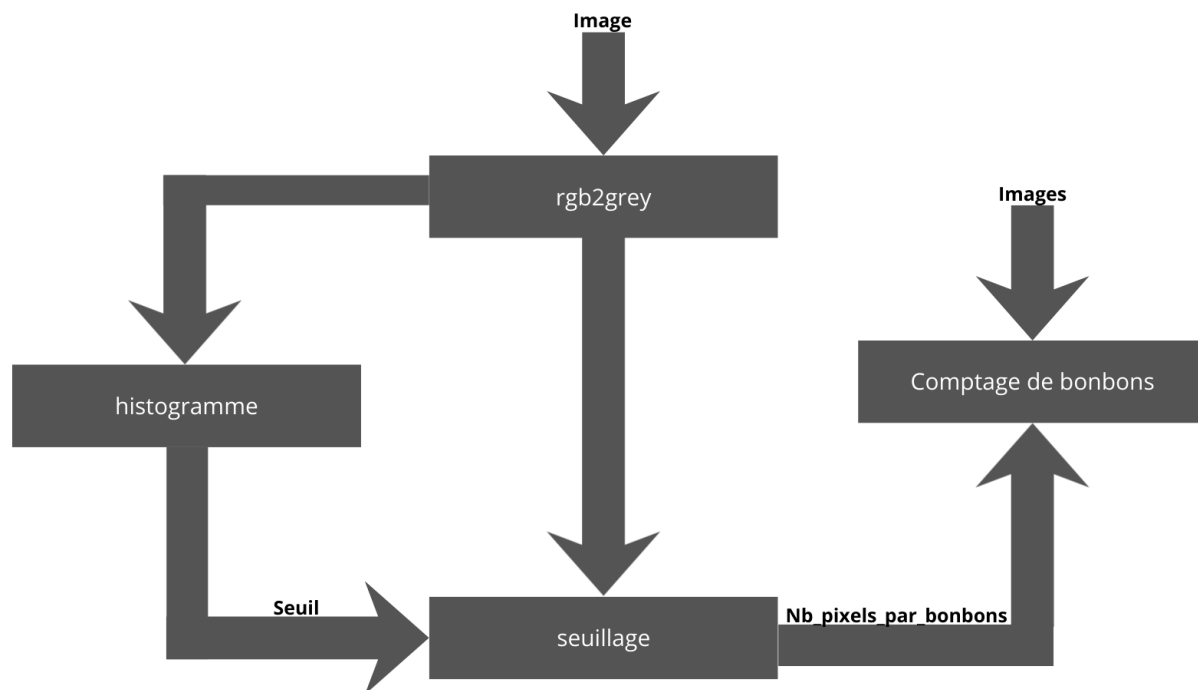


FIGURE 2.1 – Plans de réalisation

Pour la première partie, notre objectif est de compter le nombre de pixels par bonbons. Pour faciliter le comptage, nous allons nuancer la couleur puis différencier de manière binaire les bonbons et le décors via un seuillage.

Dans un second temps, nous utiliserons les valeurs de seuil et de pixels par bonbons pour automatiser le comptage sur les images de la banque de données.

2.1 Initialisation du modèle

Nous commençons par travailler avec une seule image. L'image est en couleur, on l'a passe en niveau de gris avec la fonction "rgb2grey"

```
1 def rgb2grey(img_c):  
2     img_c=img_c.astype('float')  
3     resultat=(img_c[:, :, 0]+img_c[:, :, 1]+img_c[:, :, 2])/3  
4     resultat=resultat.astype('uint8')  
5     return resultat
```

Listing 2.1 – rgb2grey : image couleur à image grise

Ainsi, les pixels obtenus sont codés en "8bits", de valeurs comprises entre 0 et 255.

Chacune des ces valeurs de pixels sont énumérées et placées dans un histogramme.

```
1 def histogramme(image):
2     l,c = image.shape[:2]
3     h=np.zeros([256,1])
4     for i in range(l) :
5         for j in range(c) :
6             val=image[i,j]
7             h[val] +=1
8     return h
```

Listing 2.2 – Histogramme : valeurs des pixels

Pour cette application, on obtient 2 pics plus ou moins large, notamment autour de ≈ 255 (bonbons blancs) et de ≈ 50 (décors noir).

On choisit une valeur de seuil pour séparer ces 2 zones de pixels. L'objectif est d'avoir une image à valeur binaire afin de compter précisément les pixels des bonbons.

```
1 def seuillage(seuil,image):
2     l,c = image.shape[:2]
3     for i in range(l) :
4         for j in range(c) :
5             val=image[i,j]
6             if val<seuil:
7                 image[i,j]=0
8             else:
9                 image[i,j]=255
10    return image
```

Listing 2.3 – Seuillage : binarisation des valeurs

Sur l'image renvoyé, on distingue clairement les différences entre les zones de pixels des bonbons et du décors. La précision de la frontière du bonbon se fait avec le choix de la valeur de seuil.

Il faut maintenant compter le nombre de pixels par bonbons. Pour se faire, on réalise un histogramme de l'image seuillée. On observe 2 pics distincts correspondant au décors et aux bonbons. On récupère la dernière valeur de l'histogramme (pixels blancs) que l'on divise par le nombre de bonbons affichées à l'image.

```
1 def Init(image,seuil) :
2     # ...
3     # Histogramme de l'image seuillee
4     h=histogramme(image1bis)
5     nb_pixels=h[255]/3
6     return nb_pixels
```

Listing 2.4 – Init : Comptage des pixels

Dans le code, je parle d'initialisation car c'est à partir de cette valeur de pixels par bonbons que l'on va pouvoir compter les bonbons de manière automatique.



FIGURE 2.2 – Image en niveau de gris

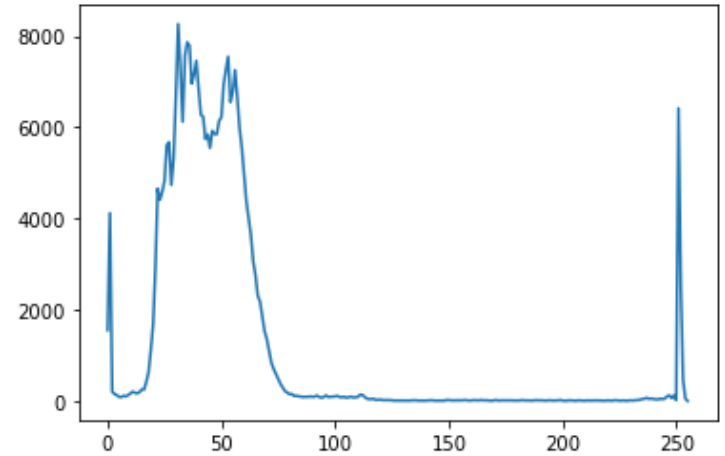


FIGURE 2.4 – Histogramme d'avant seuillage

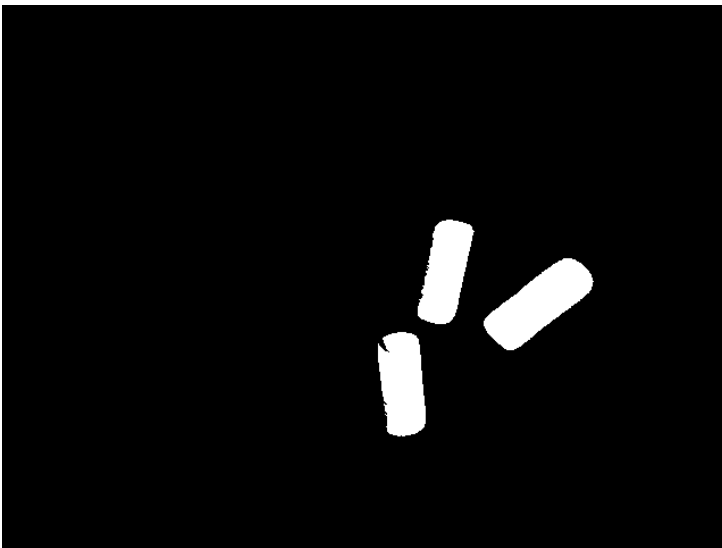


FIGURE 2.3 – Image seuillée

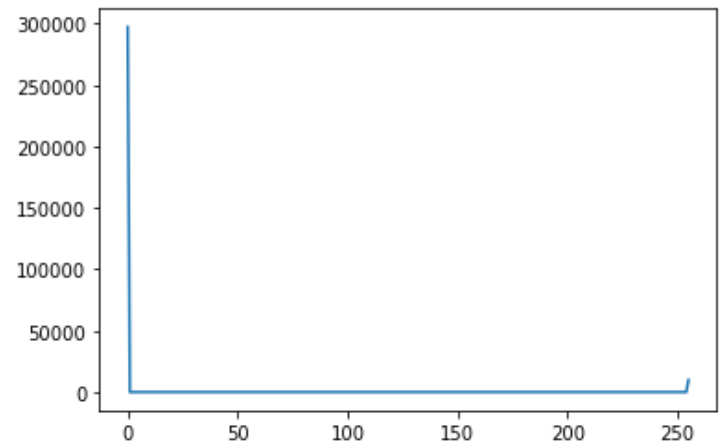


FIGURE 2.5 – Histogramme pour le comptage

On retrouve sur les images précédentes les résultats du code exécuté par les fonctions *rgb2grey()*, *histogramme()*, *seuillage()*, *Init()*.

2.2 Test du modèle

Dans cette seconde partie, nous avons suivie une phase de test pour compter les bonbons. Pour cela, on utilise une fonction similaire à celle de "Init" que l'on appelle "Test", à la différence que l'on connaît le nombre de pixels par bonbons et que le seuil est différent.

```
1 def Test(image,nb_pixels_par_bonbons,seuil):  
2     # ...  
3     #Nombre de bonbons  
4     nb_bonbons=h[255]/nb_pixels_par_bonbons
```

```

5   arrondis=round(nb_bonbons[0])
6   return nb_bonbons, arrondis

```

Listing 2.5 – Test : Comptage des bonbons

La précision du comptage dépend énormément du seuil choisit.

2.3 Analyse des résultats

Bien que la méthode soit la même, la précision des résultats diffère selon les images. Cela est dû à la variation de l'éclairage. Les pics et leur largeur changent, il faut donc trouver une valeur de seuil qui fonctionne pour toutes les images de la banque.

```

1   seuil=150 # seuil d'initialisation
2   # ...
3   seuil=90 # seuil de tests

```

Listing 2.6 – Choix des seuils

Le choix de ces valeurs s'est fait suite à de nombreux test. Il existe cependant une méthode pour trouver le meilleur seuil, on abordera cette méthode plus tard.

Voici les résultats obtenus :

```

[3304.66666667]
(array([1.19346379]), 1.0)
(array([3.17006254]), 3.0)
(array([5.20355053]), 5.0)
(array([7.22584224]), 7.0)
(array([9.06899334]), 9.0)
(array([11.07373411]), 11.0)

```

FIGURE 2.6 – Prompt Python

La première ligne correspond au nombre de pixels par bonbons réalisé lors de l'initialisation. Les ligne suivantes désignent le nombre de bonbons par image et leur approximation.

2.4 Ouverture : Méthode du cercle

Lors du seuillage, des points blancs apparaissent sur les bordures de certaines images de la banque de données. Cela est dû au reflet de la lumière sur le décors, le sol est alors légèrement éclairé et lors du seuillage ces points légèrement blancs deviennent complètement blancs.

Ce surplus de point est un problème car cela fausse le calcul du nombre de bonbons, il y en a plus que prévu. Pour contrer cela, j'ai proposé une méthode "du cercle" qui correspond en faite à de l'érosion. Cette méthode d'érosion est directement implémenté dans la fonction de seuillage de l'image :

```

1   # ...
2   rayon = 5
3   for i in range(1-rayon):

```



```

4     for j in range (c-rayon):
5         for k in range(rayon):
6             if image[i,j]==255 and image[i+k,j+k]==0 :
7                 image[i,j]=0;
8                 image[l-1-k,j+k]=0
9                 image[i+k,c-k-1]=0
10    return image
    
```

Listing 2.7 – Choix des seuils

On définit un cercle et si dans son rayon on trouve un point blanc isolé alors ce point devient noir. On fait de même sur le long des bordures de l'image mais sans le critère d'isolement car le rayon est tellement petit que si un bonbon placé sur le côté est rongé cela sera minime.

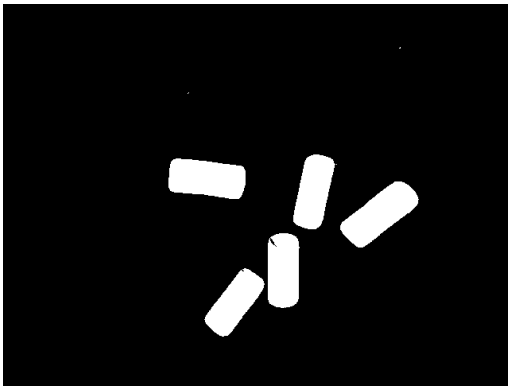


FIGURE 2.7 – Image traitée-n5

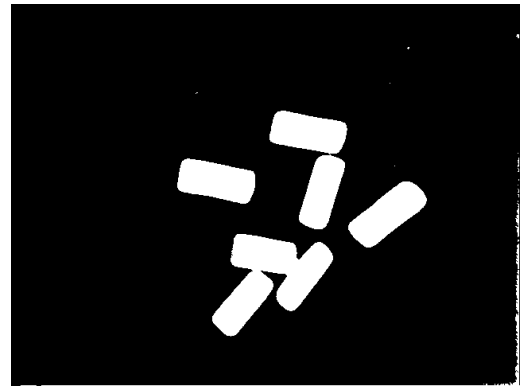


FIGURE 2.9 – Image traitée-n7

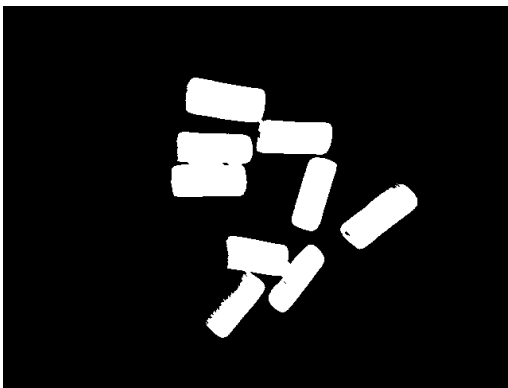


FIGURE 2.8 – Image traitée-n9



FIGURE 2.10 – Image traitée-n11

2.5 Bilan

Le traitement d'une image, nécessite au préalable une classification de ses valeurs de pixels. Il est plus facile de déterminer les frontières des objets en nuancant en gris les pixels des images. En classifiant ces valeurs de gris codé en 8 bits dans un histogrammes, on peut estimer la "quantité" de pixels. On définit alors le seuillage qui correspond à notre besoin pour binariser (0 ou 255) les pixels et effectuer des comptages d'objets ou de pixels.

3 Télédétection par drone

L'objectif de ce TP est d'évaluer l'évolution de la croissance d'un champ de betteraves au cours du temps. Pour ce faire, nous allons traiter 3 des images multispectrales acquises par drones.

Ainsi, nous devons commencer par :

- analyser une image pour déterminer le profil spectral des pixels de végétaux et de sol.
- différencier ces 2 types de pixels via un indice de végétation basé sur des profils spectraux.
- généraliser cette différenciation pour comparer les photos aux différentes dates
- seuiller et compter les pixels de végétation pour évaluer la croissance du champ.

3.1 Initialisation

L'objectif de cette partie est de localiser un pixel de sol et de végétation afin d'évaluer leur réflectance. Nous travaillons sur l'image du 24 Mai. Cette image possède 6 bandes de longueur d'ondes : $450nm$, $530nm$, $560nm$, $675nm$, $730nm$ et $850nm$.

Voici l'image affichée en nuance de gris dans les 6 bandes spectrales :

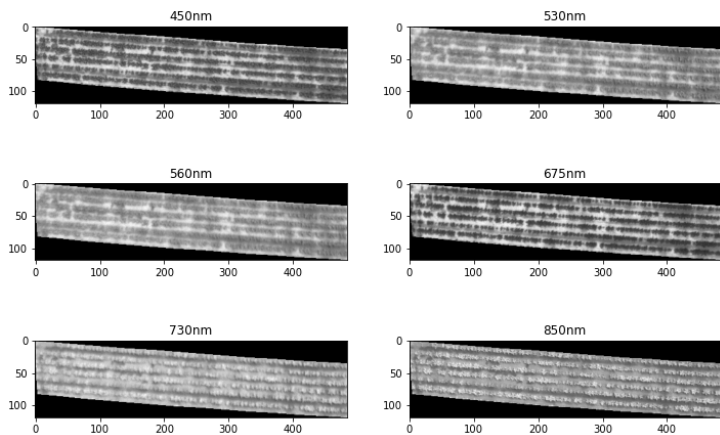


FIGURE 3.1 – Bandes en niveau de gris

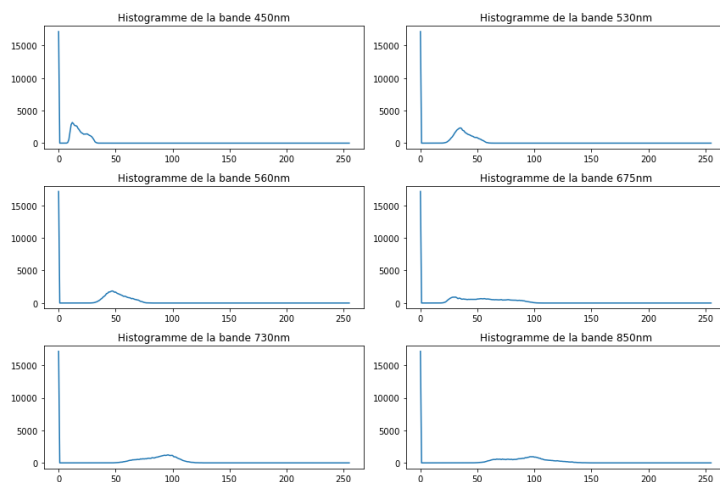


FIGURE 3.2 – Histogrammes correspondants

L'image du spectre de $450nm$ (bleu) est celui avec les pixels les plus foncés, ils désignent la végétation. On remarque que les histogrammes de la bande $675nm$ (rouge) et $850nm$ (proche infrarouge) sont opposés, leurs pics de pixels clairs et sombres sont inversés et les désignation pixels clairs-sol et pixels sombres-végétation sont inversés.

En utilisant, une composition colorée de l'image en vraies ($450nm$, $530nm$, $675nm$ - RGB) et fausses couleurs ($560nm$, $730nm$, $850nm$ - VR PIR), on observe qu'en fausse couleur la délimitation entre le sol et la végétation est beaucoup plus marquante en fausse couleur.

On localise facilement un pixel de sol en [62, 111] et de végétation en [52, 74]. Nous affichons pour chaque pixels leurs valeurs dans les différentes bandes spectrales :



FIGURE 3.3 – Profil spectral d'un pixel de betterave (vert) et de sol (bleu)

En analysant ces profil spectraux, on remarque que selon les bandes il y a des différences plus ou moins importantes entre les valeurs des pixels. Les valeurs sont très proches pour 850nm (Proche Infra Rouge) et à contrario sont le plus écartés pour 675nm (rouge).

3.2 Différentiation et Comparaison

Maintenant que nous savons les principales caractéristiques des bandes spectrales de l'image et des pixels de végétation. Nous allons calculer un indice de végétation, appelé NDVI, pour différencier les pixels.

Dans la section précédente, on a trouvé que les principales ressemblances et différences entre les pixels de sol et de végétation se font sur les bandes de proche infra rouge et de rouge. Nous allons donc construire notre indice de végétation en réutilisant ces 2 bandes.

```

1 def IndVege(image,i):
2     data=sio.loadmat(image)
3     I=data['I']
4     PIR=I[:, :, 5].astype('float')
5     R=I[:, :, 3].astype('float')
6     IV=(PIR-R)/(R+PIR)
7     # ...
8     ima=plt.imshow(IV,cmap='jet')
9     # ...

```

Listing 3.1 – Indice de végétation

Avec la formule $IV = \frac{PIR-R}{PIR+R}$, on remarque que plus la valeur est proche de 0 plus le pixel correspondra à du sol et que plus la valeur du pixel est proche de 1 plus il équivaut à de la végétation. On peut voir cela en faisant correspondre les courbes bleu et verte du graph 3.3. Le rôle du `cmap = 'jet'` est d'appliquer un gradient de couleur sur les valeurs de [0, 1].

Avec ce traitement indiciel, on pourra facilement voir les différences de végétation sur l'image.

Maintenant que nous avons les outils pour différencier la culture du reste de l'image, nous allons comparer les images à différentes dates. Pour cela, on affiche les 3 images en vraies et fausses couleurs ainsi que leurs indices de végétation.

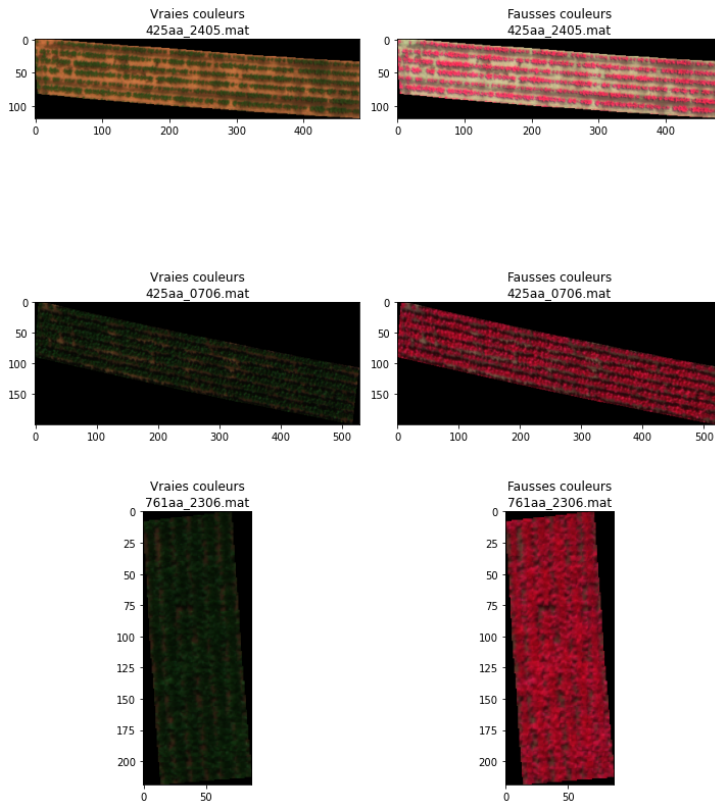


FIGURE 3.4 – Composition colorée des 3 images

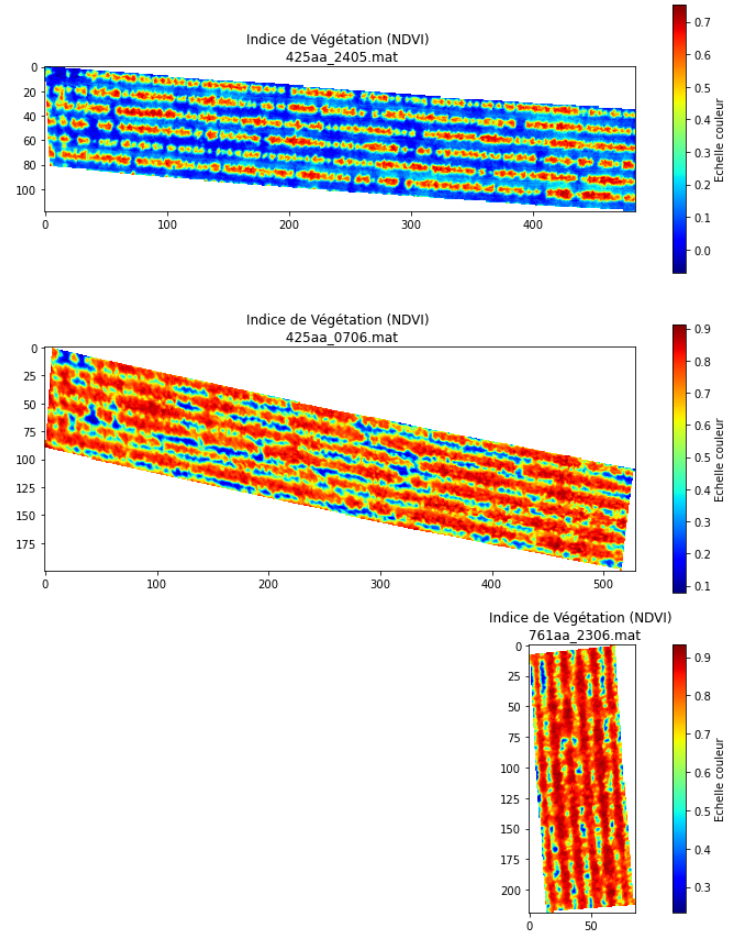


FIGURE 3.5 – Indice de végétation correspondant

La première image a peu de végétation, en couleurs réelles on voit encore les pixels de sol et en fausses couleurs la végétation est désigné par des pixels roses et non rouge. On peut confirmer tout cela en regardant l'indice de végétation correspondant, l'image est majoritairement bleu, la valeur des pixels en bande $675nm$ est élevée.

Les deux images suivantes ont été prises de nuit. En couleurs réelles on voit mal la végétation, cependant en fausses couleurs les sillages sont rouges écarlates. Cette végétation est encore plus flagrante sur les images indicielles. On remarque que plus on avance dans le temps, plus la végétation est dense.

On évalue l'intensité de la présence de la végétation via l'échelle de couleur. Cette échelle est variable selon les images. Les valeurs de cette échelle nous sera utile pour seuiller l'image.

3.3 Calcul de proportion

Nous allons maintenant chercher à seuiller et à compter les pixels de végétation pour évaluer la croissance de la culture dans le temps.

A partir des résultats obtenus précédemment, on évalue la valeur du seuil à 0.5 pour la première image, à 0.7 pour la seconde et 0.8 pour la dernière. Les seuils sont variables

Ainsi, toutes les valeurs de pixels supérieures à ces seuils seront considérées comme de la végétation et celles inférieures comme du sol.

```

1 def seuil(image,seuil,i):
2     # ...
3     IV=(PIR-R)/(R+PIR)
4     mask_veg=(IV>seuil)
5     mask_sol=(IV<seuil)&(~np.isnan(IV))
6     mask=mask_veg
7     # ...
8     ima=plt.imshow(mask)
9     nb_veg=np.sum(mask_veg)
10    nb_sol=np.sum(mask_sol)
11    prop=nb_veg/(nb_veg+nb_sol) *100
12    return ima,prop
    
```

Listing 3.2 – Fonction de seuil

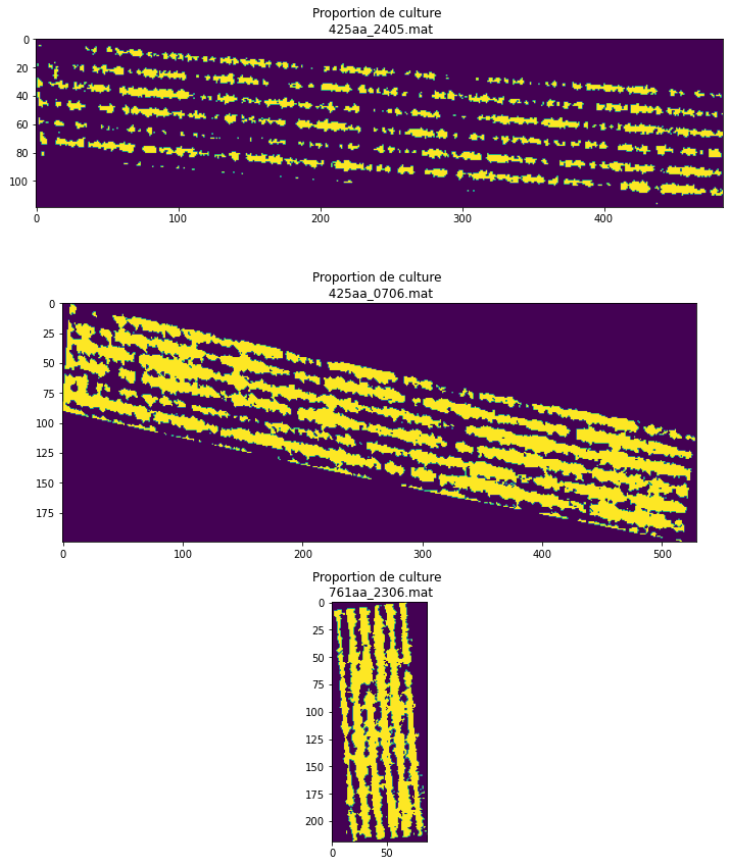


FIGURE 3.6 – Seuillage via l'indice de végétation

On évalue maintenant la proportion de culture dans le temps. Pour cela, on normalise la quantité de pixel végétal. On récupère la deuxième valeur fournie par la fonction *seuil*. On affiche cette valeur dans un graph avec le temps, celui-ci étant marqué sur le nom des images. On obtient le graph suivant :



FIGURE 3.7 – Proportion de culture

Les résultats sont cohérent, la récolte des betteraves se faisant de début juillet à août.

3.4 Bilan

La réflectance ou profils spectraux des bandes est un outil essentiel pour la différenciation d'objets sur une image. Il est possible d'utiliser la variation d'intensité entre les bandes pour distinguer des objets. Il faut parfois exploiter les longueurs d'onde du champ du visible et de l'invisible pour le traitement. En utilisant une méthode indicelle $\frac{max-min}{max+min}$, on peut échelonner les pixels selon l'intensité de leurs valeurs.

4 Cartographie des fonds par images sous-marines

L'objectif de ce TP est de cartographier des fonds marins par imagerie sous-marine. Comme les méthodes des précédents TP, nous commencerons par analyser le profil radiométrique (histogramme) des bandes de l'image pour savoir si il y a des distinctions entre les objets de l'image, puis on échantillonnera les différents objets afin d'observer leur profils spectraux qui servira ensuite de base numérique à la classification. Pour ce faire, nous allons travailler à partir de l'image suivante :

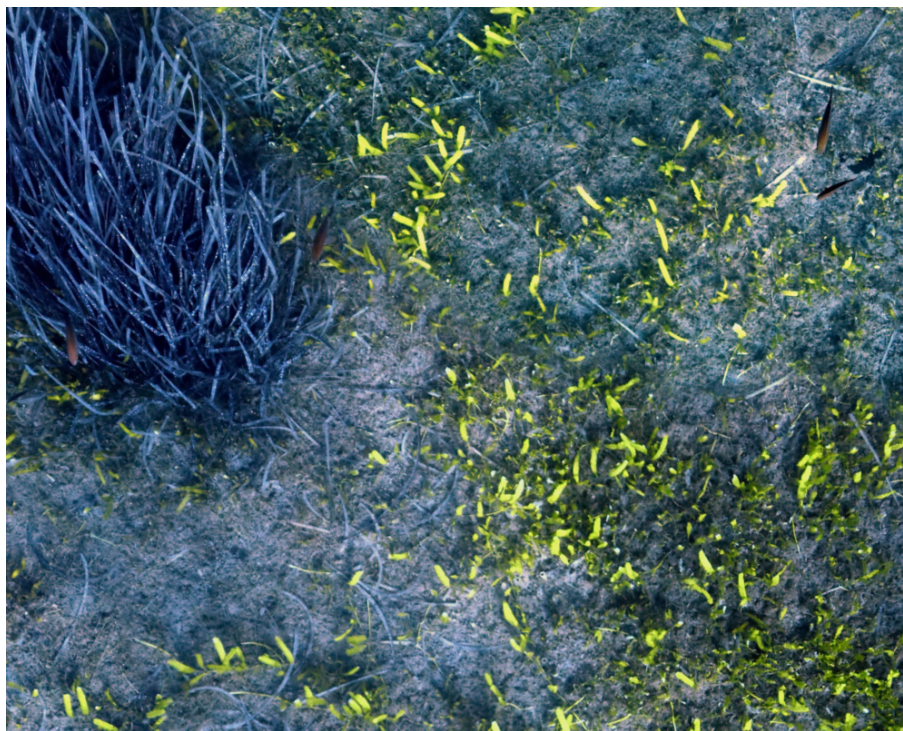


FIGURE 4.1 – Fond marin - IFREMER

On distingue 4 classes d'objets. Le repère, pour les pixels, est choisi en haut à gauche de l'image. Les différentes classes sont caractérisées par :

- une zone sombre bleu foncé/noir et dense sur la gauche en $[162, 307]$, correspondant à de la Posidonie.
- des zones de sable apparaissant en gris-bleuté/marron clair, notamment en $[201, 615]$.
- des taches vertes vives sur le sol par exemple en $[480, 626]$, correspondant à l'algue invasive Caulerpa Taxifolia.
- 4 petits poissons de couleurs beige clair et noir foncé, par exemple à gauche en $[74, 383]$.

Il y a 3 items distincts à cartographier. Les classes "posidonie", "sable" et "caulerpa". La classe "poisson" sera abordé en ouverture car son succès a été peu concluant.

Nous allons étudier si les classes (sable, posidonie et caulerpa) ont des signatures radiométriques bien différentes, ce qui facilitera leur distinction.

4.1 Affichage des bandes et histogramme

L'histogramme va servir à vérifier la séparabilité des classes avant de lancer un algorithme de classification.

Séparé l'affichage des bandes (R, G, B) permet de voir quelle longueur d'onde caractérise le mieux les éléments. Sous l'eau, l'absorption varie selon la couleur (le rouge est vite absorbé, le bleu diffuse). Par exemple, on peut constater que le canal vert fait mieux ressortir les algues que le rouge.

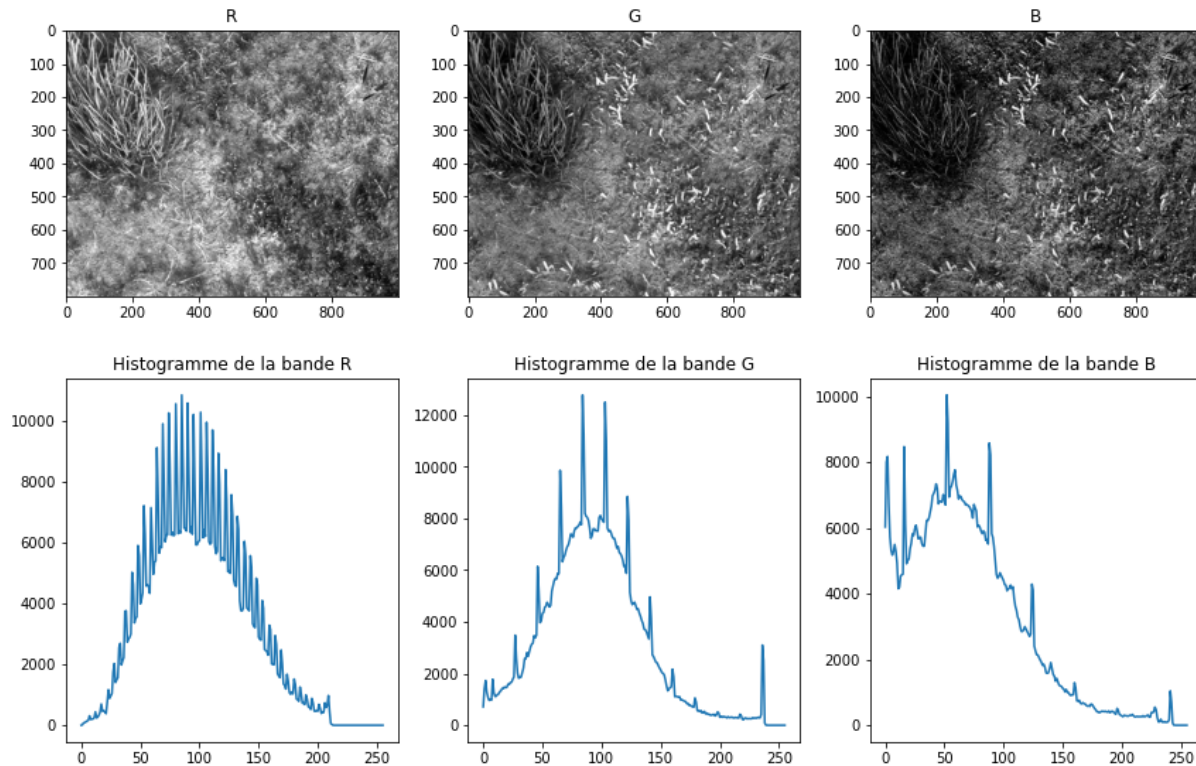


FIGURE 4.2 – Histogramme des bandes RGB

Sur chaque bande, il y a plusieurs pics, cela indique qu'il y a plusieurs objets sur l'image avec des valeurs de couleur RGB différente de celle prédominante (large bande). Ces objets se distinguent du décor par l'intensité de leur pixels dans certaines bandes. Ainsi, à partir de l'échelle, on sait que certains objets sont beaucoup plus vert que bleu ou que rouge.

L'histogramme permet d'analyser la qualité de l'image et surtout de détecter les classes : la présence de plusieurs "bosses" sur chacune des bandes, indique qu'il y a plusieurs types d'objets distincts (sable, posidonie, algue) ayant des niveaux de gris différents, ce qui confirme qu'une classification est possible.

Si il n'y avait qu'un seul bloc continu, les classes risqueraient d'être confondues dans chaque bande spectrale et une classification par niveau d'intensité des pixels par bandes serait plus difficile. Il faudrait alors utiliser d'autre méthode de classification utilisant les bords ou la texture.

4.2 Échantillonnage par région d'intérêt (ROI)

Chacune des bandes des histogrammes étant différentes, nous allons pouvoir analyser la réflectance de chacune des classes afin de les différencier numériquement. Pour ce faire, on sélectionne pour chaque classe des échantillons en formant un rectangle autour des régions d'intérêt. Nous calculons alors les valeurs moyennes de pixels pour chaque bande RGB.

Lors de la compilation du code, les zones ROI ne sont jamais parfaitement identiques ce qui peut créer des variations sur la valeur moyenne obtenue. Nous allons afficher les profils spectraux (réflectance) des différentes classes. Nous allons étudier si la classe poisson peut être étudiée

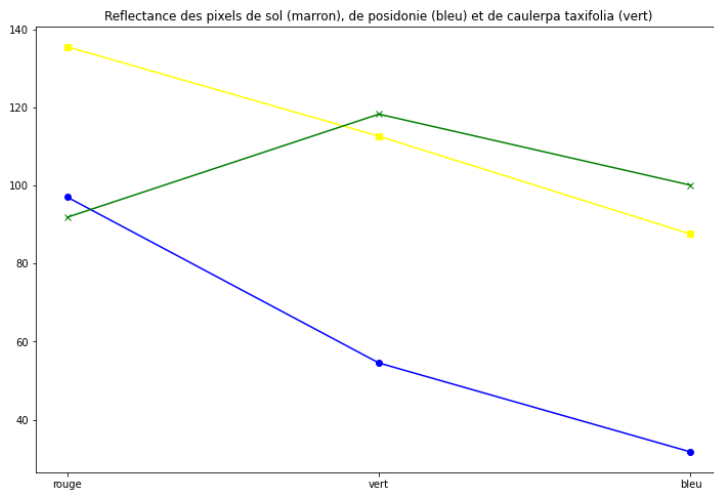


FIGURE 4.3 – Réflectance des classes sables (en jaune), posidonie (en bleu) et caulerpa (en vert)

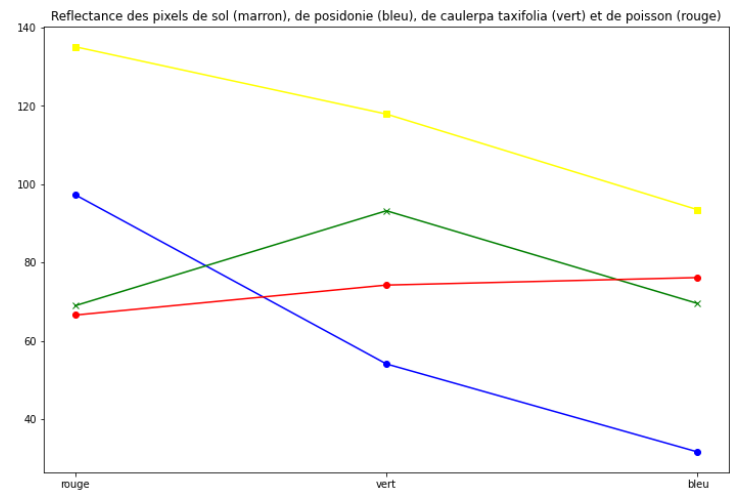


FIGURE 4.4 – Réflectance avec la classe poisson (en rouge)

Les profils des différentes classes restent plus ou moins le même, cependant selon les compilations l'intensité des valeurs varie. Sur la deuxième figure, les pixels de l'échantillon de la Caulerpa Taxifolia ont des valeurs plus basse que lors du première échantillonnage. Cette différence peut s'expliquer par le degrés de précision lors de la sélection du ROI. Plus la zone de sélection est petite et ne prend que l'algue de Caulerpa sans prendre le décors, plus la valeur valeur moyenne sera haute.

Le but de tracer les profils spectraux des classes et de vérifier que ceux-ci ne se chevauchent pas. Plus les différences entre les profils des classes sera élevé plus il sera facile de classer les objets de l'image.

Sur la première figure, la réflectance du sable et de la caulerpa pour les bandes verte et bleu est très très similaire. Il faudra donc prendre en compte la bande rouge pour différencier le sable de la caulerpa. A contrario, la réflectance de la posidonie et de la caulerpa est très proche sur la bande rouge, pour différencier ces 2 classes il faudra donc se baser sur les bandes verte et bleu. On en conclue que toute les bandes seront importantes pour la classification et qu'il faudra travailler avec les 3 valeurs moyennées des bandes.

Sur la deuxième figure, le profil spectral de la classe poisson est extrêmement proche de celle de la Caulerpa. On s'attend à ce que lors de la classification qu'il y ait des erreurs de détection entre les algues Caulerpa Taxifolia et les poissons.

4.3 Classification

Pour appliquer la classification à l'image, il faut associer chaque pixel à la classe la plus proche du graphique des profils spectraux. Pour cela, nous allons tester 2 méthodes de calcul de distance :

— la distance euclidienne :

$$d(x, r_k) = \sqrt{\sum_{i=1}^N (x(i) - \bar{r}_k(i))^2}$$

— la spectral angle mapper :

$$\alpha(x, r_k) = \cos^{-1} \left(\frac{\sum_{i=1}^N x(i) * \bar{r}_k(i)}{\sqrt{\left(\sum_{i=1}^N x(i)^2\right)} * \sqrt{\left(\sum_{i=1}^N \bar{r}_k(i)^2\right)}} \right)$$

Voici les figures qui sont obtenues :

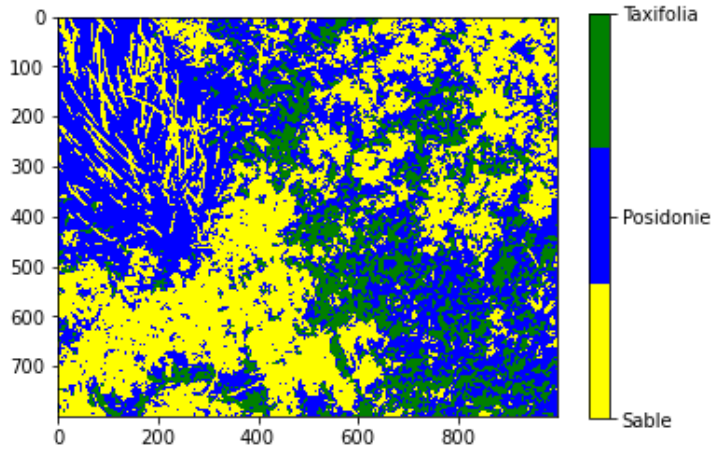


FIGURE 4.5 – Classification via la méthode euclidienne

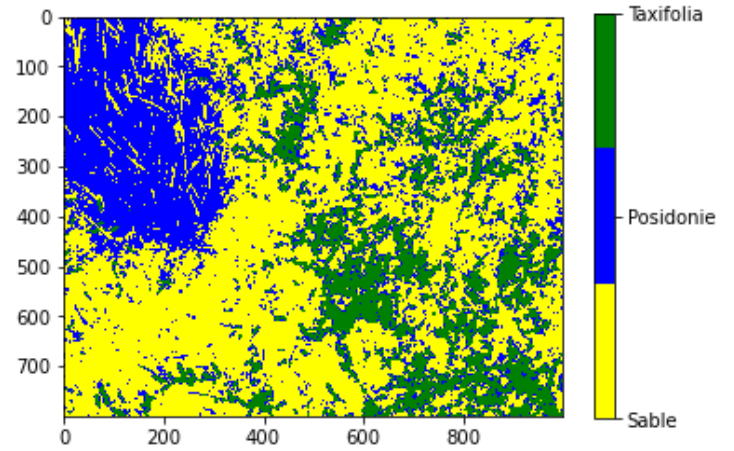


FIGURE 4.6 – Classification via la méthode d'angle

La distance donnant les meilleurs résultats et celle du Spectral Angle Mapper (SAM), cela s'explique par le fait que le SAM est insensible aux variations d'illumination.

Contrairement à la distance euclidienne, le SAM ne regarde que la forme du spectre (l'angle) et ignore son amplitude (la luminosité). De plus, il reconnaît qu'un pixel à l'ombre et un pixel au soleil sont constitués du même matériau, alors que la distance euclidienne les classerait souvent comme deux matériaux différents.

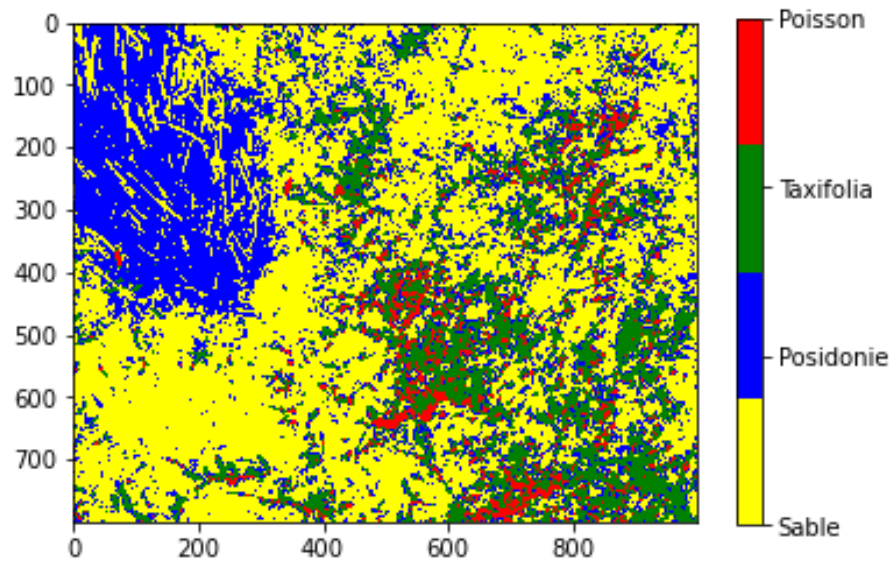


FIGURE 4.7 – Classification via la méthode d'angle des classes avec le poisson

Nous nous intéresserons à la méthode de distance SAM pour la classification avec le poisson car l'image obtenue avec la méthode euclidienne possède trop de défauts.

Les zones de l'image où apparaissent des poissons sont mises en rouges, cependant sur le milieu-bas de l'image, on voit que les algues de *Caulerpa Taxifolia* sont associées à des poissons. Cela est dû au fait que le profil spectral du poisson est trop proche de celui de la caulerpa. Avec juste une méthode d'analyse des couleurs RGB des pixels, on ne peut pas dissocier clairement ces 2 classes.

4.4 Bilan

La classification d'une image se fait de manière supervisée. Il faut échantillonner des régions d'intérêts (ROI) et comparer la valeur moyenne des pixels aux profils spectraux des bandes. Cette comparaison se fait par un calcul de distance, la méthode spectral angle mapper (SAM) se base sur la forme du spectre ce qui permet d'ignorer l'intensité des pixels et de mieux gérer les cas avec des ombres, réduisant les erreurs. Cette différenciation par profil spectral peut se faire car les bandes et les piques des histogrammes sont différents, si cela n'avait pas été le cas on aurait dû se baser sur les bords et la texture des objets.

5 Conclusion

Les travaux présentés dans ce rapport ont permis d'explorer la chaîne complète de traitement d'images, de l'acquisition de la donnée brute à l'extraction d'informations sémantiques, à travers trois cas d'usage distincts.

Pour la vision industrielle, nous avons validé une méthode de comptage automatique basé sur l'analyse d'histogrammes et le seuillage binaire. Les tests ont mis en évidence la sensibilité de cette approche aux variations d'éclairage et aux reflets. L'implémentation de la méthode dite "du cercle" (érosion) s'est avérée efficace pour éliminer le bruit et isoler les objets, garantissant ainsi une meilleure robustesse du comptage.

Dans le cadre de la télédétection agricole, l'exploitation de l'imagerie multispectrale a démontré la nécessité d'utiliser des bandes au-delà du visible (proche infrarouge) pour le suivi biologique. Le calcul de l'Indice de Végétation (NDVI) a permis de segmenter précisément la biomasse par rapport au sol et de quantifier la croissance des cultures avec une cohérence temporelle.

Enfin, la cartographie marine a souligné l'importance du choix de l'algorithme de classification. La comparaison entre la distance euclidienne et le Spectral Angle Mapper (SAM) a montré la supériorité du SAM pour les environnements sous-marins. En se basant sur la forme du profil spectral plutôt que sur l'amplitude, le SAM s'affranchit des atténuations lumineuses (ombres) du milieu aquatique. Cependant, la confusion persistante entre certaines classes (poissons et algues *Caulerpa*) rappelle les limites de la classification purement radiométrique lorsque les signatures spectrales sont trop proches.

En conclusion, ces expérimentations confirment que la fiabilité d'un système de vision repose autant sur les bandes spectrales captées que sur la capacité des algorithmes (seuillage adaptatif, SAM) à compenser les différentes variations causées par l'environnement.

6 Annexe

6.1 Code Vision Industrielle

```
1  # -*- coding: utf-8 -*-
2  import numpy as np
3  import cv2
4  import matplotlib.pyplot as plt
5
6  # Niveaux de gris
7  def rgb2grey(img_c):
8      img_c=img_c.astype('float')
9      resultat=(img_c[:, :, 0]+img_c[:, :, 1]+img_c[:, :, 2])/3
10     resultat=resultat.astype('uint8')
11     return resultat
12
13 # Affichage
14 def affiche(image):
15     cv2.imshow("image", image)
16     cv2.waitKey(0)
17     cv2.destroyAllWindows()
18
19 # Histogramme
20 def histogramme(image):
21     l,c = image.shape[:2]
22     h=np.zeros([256,1])
23     for i in range(l) :
24         for j in range(c) :
25             val=image[i,j]
26             h[val] +=1
27     return h
28
29 # Seuillage
30 def seuillage(seuil, image):
31     l,c = image.shape[:2]
32     for i in range(l) :
33         for j in range(c) :
34             val=image[i,j]
35             if val<seuil:
36                 image[i,j]=0
37             else:
38                 image[i,j]=255
39
40     rayon = 5
41     for i in range(l-rayon):
42         for j in range(c-rayon):
43             for k in range(rayon):
44                 if image[i,j]==255 and image[i+k,j+k]==0 :
45                     image[i,j]=0;
46
47                 image[l-1-k,j+k]=0
48                 image[i+k,c-k-1]=0
```

```

49         # if image[i,j]==0 and image[i+k,j+k]==255 :
50         #     image[i,j]=255;
51
52
53     return image
54
55
56
57 #Initialisation pour taille en pixels des bonbons
58 def Init(image,seuil) :
59     # fonction pour import et afficher la taille ...
60     image1 = cv2.imread('test_3bb_blancs.png',1)
61     l,c,b = image1.shape[:3]
62     print(l,c,b)
63
64     # 1
65     # Visualiser l'image
66     #affiche(image1)
67     image1bis=rgb2grey(image1)
68     #affiche(image1bis)
69     # Visualiser son histogramme
70     h=histogramme(image1bis)
71     plt.plot(h)
72     plt.show()
73     plt.close("all")
74
75     # 2
76     image2=seuillage(seuil,image1bis)
77     affiche(image2)
78     # Visualiser son histogramme
79     h=histogramme(image2)
80     plt.plot(h)
81     plt.show()
82     plt.close("all")
83
84
85     #Nombre de pixels
86     nb_pixels=h[255]/3
87     return nb_pixels
88 seuil=150#157
89 nb_pixels_par_bonbons=Init('test_3bb_blancs.png',seuil)
90 print(nb_pixels_par_bonbons)
91
92
93 #Phase de Test de comptage de bonbons
94 def Test(image,nb_pixels_par_bonbons,seuil):
95     # fonction pour import et afficher la taille ...
96     image1 = cv2.imread(image,1)
97
98     # 1
99     # Visualiser l'image
100     #affiche(image1)
101     image1bis=rgb2grey(image1)
102     #affiche(image1bis)
103     # Visualiser son histogramme
104     h=histogramme(image1bis)

```

```

105     plt.plot(h)
106     plt.show()
107     plt.close("all")
108
109     # 2
110     image2=seuillage(seuil,image1bis)
111     affiche(image2)
112     # Visualiser son histogramme
113     h=histogramme(image2)
114     plt.plot(h)
115     plt.show()
116     plt.close("all")
117
118
119     #Nombre de pixels
120     nb_bonbons=h[255]/nb_pixels_par_bonbons
121     arrondis=round(nb_bonbons[0])
122     return nb_bonbons,arrondis
123
124 seuil=90
125 C=Test('test_1bb_blancs.png',nb_pixels_par_bonbons,seuil)
126 print(C)
127
128 C=Test('test_3bb_blancs.png',nb_pixels_par_bonbons,seuil)
129 print(C)
130
131 C=Test('test_5bb_blancs.png',nb_pixels_par_bonbons,seuil)
132 print(C)
133
134 C=Test('test_7bb_blancs.png',nb_pixels_par_bonbons,seuil)
135 print(C)
136
137 C=Test('test_9bb_blancs.png',nb_pixels_par_bonbons,seuil)
138 print(C)
139
140 C=Test('test_11bb_blancs.png',nb_pixels_par_bonbons,seuil)
141 print(C)

```

Listing 6.1 – Vision Industrielle

6.2 Code Télédétection par Drones

```

1  # -*- coding: utf-8 -*-
2  import numpy as np
3  import cv2
4  import matplotlib.pyplot as plt
5  import scipy.io as sio
6
7
8  #Partie1 - Affichage
9  def affiche(image):
10     data=sio.loadmat(image)
11     I=data['I']
12     dim=I.shape
13     plt.figure(figsize=(12,8))
14     titres=['450nm','530nm','560nm','675nm','730nm','850nm']
15     for i in range(dim[2]):
16         plt.subplot(3,2,i+1)
17         plt.imshow(I[:, :, i], cmap='gray')
18         plt.title(titres[i])
19     plt.show()
20
21
22 #Partie1 -Composition coloree
23 #On a multiplie par 2 les valeurs pour avoir du plus clair
24 def VFcouleurs(image,ligne):
25     data=sio.loadmat(image)
26     I=data['I']
27     #plt.figure(figsize=(12,8))
28     plt.subplot(3,2,ligne*2+1)
29     plt.imshow(2*I[:, :, [3,1,0]])
30     plt.title(f'Vraies couleurs\n{image}')
31     plt.subplot(3,2,ligne*2+2)
32     plt.imshow(2*I[:, :, [5,3,2]])
33     plt.title(f'Fausses couleurs\n{image}')
34     #plt.show()
35
36
37 #Partie1 -histo
38 def HistoCosto(image):
39     data = sio.loadmat(image)
40     I = data['I']
41     I=I.astype(np.float32)
42     dim=I.shape
43     plt.figure(figsize=(12,8))
44     titres=['450nm','530nm','560nm','675nm','730nm','850nm']
45     for i in range(dim[2]):
46         plt.subplot(3,2,i+1)
47         h = cv2.calcHist([I], [i], None, [256], [0, 1])
48         plt.plot(h)
49         plt.title('Histogramme de la bande ' + titres[i])
50     plt.tight_layout()
51
52
53 #Partie1 - valeur des pixels de sol et de vegetal

```

```

54 def Valpix(image):
55     data=sio.loadmat(image)
56     I=data['I']
57     dim=I.shape
58     val1=[]
59     val2=[]
60     for i in range(dim[2]):
61         val1.append(I[62,111,i],)
62         val2.append(I[52,74,i])
63     longueuronde=[450,530,560,675,730,850]
64     plt.figure(figsize=(12,8))
65     plt.plot(longueuronde,val1, color='blue', marker='s')
66     plt.plot(longueuronde,val2, color='green', marker='o')
67     plt.title("Profils spectraux (reflectance) d'un pixel de bettrave (vert) et de sol (
bleu)")
68     plt.show()
69     return val1, val2
70
71
72 #Partie2 - Calcul du NDVI
73 def IndVege(image,i):
74     data=sio.loadmat(image)
75     I=data['I']
76     PIR=I[:, :, 5].astype('float')
77     R=I[:, :, 3].astype('float')
78     IV=(PIR-R)/(R+PIR)
79     # plt.figure(figsize=(12,8))
80     plt.subplot(3, 1, i + 1)
81     ima=plt.imshow(IV,cmap='jet')
82     plt.colorbar(label='Echelle couleur')
83     plt.title(f'Indice de Vegetation (NDVI) \n {image}')
84     # plt.show()
85     return ima,IV
86
87
88 #Partie 4 - Seuillage
89 def seuil(image,seuil,i):
90     data=sio.loadmat(image)
91     I=data['I']
92     PIR=I[:, :, 5].astype('float')
93     R=I[:, :, 3].astype('float')
94     IV=(PIR-R)/(R+PIR)
95     mask_veg=(IV>seuil)
96     mask_sol=(IV<seuil)&(~np.isnan(IV))
97     mask=mask_veg#+mask_sol
98     # plt.figure(figsize=(12,8))
99     plt.subplot(3, 1, i + 1)
100     ima=plt.imshow(mask)
101     plt.title(f'Proportion de culture \n {image}')
102     # plt.show()
103     nb_veg=np.sum(mask_veg)
104     nb_sol=np.sum(mask_sol)
105     prop=nb_veg/(nb_veg+nb_sol) *100
106     return ima,prop
107
108

```

```

109 # -----1 Analyse d'une image-----
110
111 affiche('425aa_2405.mat')
112 VFcouleurs('425aa_2405.mat',1)
113 HistoCosto('425aa_2405.mat')
114 Valpix('425aa_2405.mat')
115
116 # -----2 Indice de vegetation-----
117
118 A=IndVege('425aa_2405.mat',0)
119
120 # -----3 Comparaison-----
121
122 liste_image = ['425aa_2405.mat', '425aa_0706.mat', '761aa_2306.mat']
123 plt.figure(figsize=(10, 12))
124 for i, nom_image in enumerate(liste_image):
125     VFcouleurs(nom_image, i)
126 plt.tight_layout()
127 plt.show()
128
129
130 plt.figure(figsize=(10, 12))
131 derniere_image = None
132 for i, nom_image in enumerate(liste_image):
133     derniere_image = IndVege(nom_image, i)[0]
134 plt.tight_layout()
135 plt.show()
136
137 # -----4 Proportion de culture-----
138
139 plt.figure(figsize=(10, 12))
140 proportions=[]
141 listeseuil=[0.5,0.7,0.8]
142 for i,nom_image in enumerate(liste_image):
143     Image=seuil(nom_image,listeseuil[i], i)
144     proportions.append(Image[1])
145 plt.tight_layout()
146 plt.show()
147
148 print(proportions)
149 temps=['2405','0706','2306']
150
151 plt.figure(figsize=(12,8))
152 plt.plot(temps,proportions, color='green', marker='s')
153 plt.title("Proportion de culture au cours du temps")
154 plt.show()

```

Listing 6.2 – Télédétection par drones

6.3 Code Cartographie Marine

```

1  # -*- coding: utf-8 -*-
2  import numpy as np
3  import cv2
4  import matplotlib.pyplot as plt
5  import scipy.io as sio
6  from matplotlib import colors
7
8
9  #-----Partie1-----
10 def Affiche(image):
11     im=cv2.imread(image)
12     cv2.imshow("image",im)
13     cv2.waitKey(0)
14     cv2.destroyAllWindows()
15     taille = im.shape
16     print(taille)
17     return im
18
19
20 #-----Partie2 - Affichages bandes-----
21 def HistoBande(image):
22     data=cv2.imread(image)
23     I=data
24     dim=I.shape
25     plt.figure(figsize=(12,8))
26     titres=['R','G','B']
27     for i in range(dim[2]):
28         plt.subplot(2,3,i+1)
29         plt.imshow(I[:, :, i], cmap='gray')
30         plt.title(titres[i])
31         plt.subplot(2,3,i+4)
32         l,c = dim[:2]
33         h=np.zeros([256,1])
34         for j in range(l) :
35             for k in range(c) :
36                 val=data[j,k,i]
37                 h[val] +=1
38         plt.plot(h)
39         plt.title('Histogramme de la bande ' + titres[i])
40     plt.tight_layout()
41     plt.show()
42
43
44 #-----Partie 3-----
45 def Moyenne(c,l,w,h,image):
46     im=cv2.imread(image)
47     dim=im.shape
48     liste=[]
49     for j in range(dim[2]):
50         M=np.mean(im[l:l+h,c:c+w,j])
51         liste.append(M)
52     return liste
53

```

```

54
55
56 def ROI(image):
57     im=cv2.imread(image)
58     bande=['rouge','vert','bleu']
59     # ROI=['sable','posidonie','taxifolia','poisson']
60     roi=['sable','posidonie','taxifolia']
61     echantillon=[]
62     val=[]
63     for i in enumerate(roi):
64         r=cv2.selectROI(im)
65         cv2.destroyAllWindows()
66         l,c,w,h=r
67         val.append(Moyenne(l,c,w,h,image))
68         echantillon.append(r)
69         print(echantillon)
70         print(val)
71     plt.figure(figsize=(12,8))
72     plt.plot(bande,val[0], color='yellow', marker='s')
73     plt.plot(bande,val[1], color='blue', marker='o')
74     plt.plot(bande,val[2], color='green', marker='x')
75     # plt.plot(bande,val[3], color='red', marker='o')
76     plt.title("Reflectance des pixels de sol (marron), de posidonie (bleu) et de
caulerpa taxifolia (vert)")
77     plt.show()
78     return echantillon,val
79
80
81 #methode1
82 def euclidienne(image,moyenne):
83     im=image.astype(np.float32)
84     A=im-moyenne
85     B=np.linalg.norm(A,axis=2)
86     return B
87
88
89 #methode 2
90 def anglemapper(image,moyenne):
91     im=image.astype(np.float32)
92     A=np.sum(im * moyenne,axis=2)
93     B=np.linalg.norm(im,axis=2)
94     C=np.linalg.norm(moyenne)
95     # denom = B * C
96     # denom[denom == 0] = 1e-10
97     # cos_angle = np.clip(A / denom, -1, 1)
98     cos_angle=A/(B*C)
99     D = np.arccos(cos_angle)
100    return D
101
102
103
104 def classification(image,moyenne,methode):
105     image=cv2.imread(image)
106     # roi=['sable','posidonie','taxifolia','poisson']
107     roi=['sable','posidonie','taxifolia']
108     d=[]

```

```

109     for i,nom in enumerate(roi):
110         if methode==1:
111             distance=euclidienne(image,moyenne[i])
112         else:
113             distance=anglemapper(image,moyenne[i])
114         d.append(distance)
115     total = np.dstack(d)
116     resultat = np.argmin(total, axis=2)
117     return resultat
118
119
120
121
122 # -----Execution-----
123
124 #Partie1
125 IM = Affiche('image fond ifremer.tif')
126
127 # Partie 2
128 HistoBande('image fond ifremer.tif')
129
130 # Partie3
131 moyenne=ROI('image fond ifremer.tif')
132 # cmap_perso = colors.ListedColormap(['yellow', 'blue', 'green', 'red'])
133 cmap_perso = colors.ListedColormap(['yellow', 'blue', 'green'])
134
135 carte1=classification('image fond ifremer.tif',moyenne[1],1)
136 plt.imshow(carte1,cmap_perso)
137 # cbar = plt.colorbar(ticks=[0, 1, 2, 3])
138 cbar = plt.colorbar(ticks=[0, 1, 2])
139 # cbar.ax.set_yticklabels(['Sable', 'Posidonie', 'Taxifolia', 'Poisson'])
140 cbar.ax.set_yticklabels(['Sable', 'Posidonie', 'Taxifolia'])
141 plt.show()
142
143
144 carte2=classification('image fond ifremer.tif',moyenne[1],2)
145 plt.imshow(carte2,cmap_perso)
146 # cbar = plt.colorbar(ticks=[0, 1, 2, 3])
147 cbar = plt.colorbar(ticks=[0, 1, 2])
148 # cbar.ax.set_yticklabels(['Sable', 'Posidonie', 'Taxifolia', 'Poisson'])
149 cbar.ax.set_yticklabels(['Sable', 'Posidonie', 'Taxifolia'])
150 plt.show()
    
```

Listing 6.3 – Cartographie marine